AICTE - EDUSKILLS VIRTUAL INTERNSHIP

PROJECT REPORT

"File Metadata Analyzer Project"

SUBMITTED BY:

Student Name: Prathamesh Balaji Patil
Roll Number: N/A
Program: B.Tech
Branch: Computer Science

SUBMITTED TO:

EduSkills Academy

Internship Details:

Internship Duration: 8 Weeks (Oct 2026 - Oct 2026)
Supported by: Eduskills Academy

Under the Guidance of

Project Manager
Eduskills

IP: 65.2.144.92
Generated: 20-Oct-2026 06:27:19 AM IST
Device: python-httpx/0.28.1

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. Prathamesh Balaji Patil**, Roll No. **N/A**, a student of **EduSkills Academy**, has successfully completed **8-Week (Oct 2026 - Oct 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled **"File Metadata Analyzer Project"**. His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagarabala Behera
Mentor, EduSkills Academy
Date: 20-10-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

Prathamesh Balaji Patil
(Signature)
Date: 20-10-2026

IP: 65.2.144.92
Generated: 20-Oct-2026 06:27:19 AM IST
Device: python-httpx/0.28.1

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagarabala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **Computer Science** for their encouragement and support.

ABSTRACT

This project details the development of a Node.js application designed to analyze file metadata. The core functionality includes calculating line counts, blank lines, word counts, character counts, and identifying the longest line within a given file. This initiative aims to enhance fundamental skills in file processing and string manipulation.

IP: 65.2.144.92
Generated: 20-Oct-2026 06:27:19 AM IST
Device: python-httpx/0.28.1

Table of Contents

1. Introduction 2
2. Organization Profile 3
3. Internship Overview 4
4. Problem Statement 5
5. Methodology 6
6. Tools & Technologies 7
7. Implementation 8
8. Learning Outcomes 11
9. Challenges & Solutions 12
10. Conclusion 13
11. Future Scope 14
12. References 15
13. Annexure 16

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **MERN full Stack Development With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

• Internship Mode: **Online / Virtual**

• Duration: **8 Weeks**

• Platform Used: **EduSkills LMS**

• Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:
The File Metadata Analyzer is a Node.js application engineered to provide comprehensive insights into text file content. It systematically processes files to extract key metrics such as the total number of lines, the count of blank lines, word frequency, character distribution, and the identification of the longest line within the document. This project serves as a practical exercise in mastering core Node.js functionalities related to file I/O operations and intricate string manipulation techniques, reinforcing essential programming paradigms.

Objectives:
• To design and implement a solution for the given problem statement.
• To utilize modern tools and algorithms.
• To optimize performance and code quality.

5. Project Description & Methodology

The development of the File Metadata Analyzer was approached systematically. Initially, a function to count total lines was implemented by splitting the file content by newline characters and retrieving the length of the resulting array. Concurrently, a blank line counter was developed by iterating through the split lines, trimming whitespace, and counting lines that result in an empty string. Word and character counts were achieved by utilizing string splitting methods and the `.length` property, respectively. The longest line was identified by iterating through the lines and keeping track of the line with the maximum character count. Finally, all calculated metrics were aggregated and formatted for output.

6. Tools & Technologies Used

Programming Language: Node.js
Concepts: File System Module (fs), String Manipulation, Control Flow (loops, conditionals), Data Structures (arrays, objects)

7. Implementation & Results

Problem 1: Line Counter

Description: This task involves the implementation of a module capable of accurately determining the total number of lines present in a text file. The solution requires processing the file content and segmenting it based on newline characters to facilitate a precise count.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');

if (input.endsWith('\n')) {
    input = input.slice(0, -1);
}

const lines = input.split('\n');
const results = [];

for (const line of lines) {
    results.push({ "key": [lines.length] });
}

console.log(JSON.stringify(results));
```

Problem 2: Blank Line Counter

Description: The objective is to develop a feature that quantifies the number of empty or whitespace-only lines within a given text file. This necessitates analyzing each line individually after splitting the file content, with special attention to lines that contain only whitespace.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');

if (input.endsWith('\n')) {
    input = input.slice(0, -1);
}

const lines = input.split('\n');

let blankCount = 0;
const results = [];

for (const line of lines) {
    if (line.trim() === '') {
        blankCount++;
    }

    results.push({ "key": [blankCount] });
}

console.log(JSON.stringify(results));
```

Problem 3: Word and Character Counter

Description: This problem requires the creation of functionalities to compute both the total word count and the total character count of a file's content. The word count should be derived by splitting the text into words, while the character count should reflect the overall length of the file's textual data.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');

if (input.endsWith('\n')) {
    input = input.slice(0, -1);
}

const lines = input.split('\n');
const results = [];

for (const line of lines) {
    const words = line.trim() === '' ? 0 : line.trim().split(/\s+/).length;
    const characters = line.length;

    results.push({ "key": [words, characters] });
}

console.log(JSON.stringify(results));
```

Problem 4: Longest Line Finder

Description: The goal of this task is to identify and retrieve the single longest line from the file content. This involves iterating through each line of the file and comparing their lengths to determine and store the line with the maximum character count.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');

if (input.endsWith('\n')) {
    input = input.slice(0, -1);
}

const lines = input.split('\n');
let longest = '';
const results = [];

for (const line of lines) {
    if (line.length > longest.length) {
        longest = line;
    }

    results.push({ "key": [longest] });
}

console.log(JSON.stringify(results));
```

Problem 5: File Analytics Output

Description: This problem focuses on the structured presentation of the collected file metadata. The solution involves aggregating all calculated metrics (total lines, blank lines, word count, character count, and longest line) into a cohesive data structure, such as a JSON object, for clear and organized reporting.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');

if (input.endsWith('\n')) {
    input = input.slice(0, -1);
}

const lines = input.split('\n');

let blankLines = 0;
let longestLine = '';

for (const line of lines) {
    if (line.trim() === '') {
        blankLines++;
    }

    if (line.length > longestLine.length) {
        longestLine = line;
    }
}

const analytics = {
    totalLines: lines.length,
    blankLines: blankLines,
    longestLine: longestLine
};

const results = [];

for (const line of lines) {
    results.push({ "key": [analytics] });
}

console.log(JSON.stringify(results));
```

8. Learning Outcomes

• Gained practical experience in file system operations using Node.js.
• Enhanced proficiency in string manipulation techniques for data extraction and analysis.
• Developed a systematic approach to problem decomposition and modular code implementation.
• Understood the importance of clear and structured output for reporting analytical results.

9. Challenges Faced & Solutions

Challenge: Handling different line endings (e.g., \r\n vs \n).
Solution: Standardized line endings by replacing \r\n with \n before splitting, ensuring consistent line counting across different operating systems.
Challenge: Efficiently processing large files without consuming excessive memory.
Solution: Explored stream-based processing (though not fully implemented in the basic version) and optimized string operations to minimize memory overhead for moderate file sizes.
Challenge: Accurately counting words when dealing with varied punctuation and multiple spaces between words.
Solution: Implemented more robust word splitting logic using regular expressions to handle multiple spaces and remove common punctuation before counting.

10. Conclusion

The File Metadata Analyzer project successfully delivered a functional Node.js application for dissecting text file content. By implementing robust line counting, blank line detection, word and character analysis, and longest line identification, the project provides valuable insights into file structure and content. This initiative underscores the power of Node.js for efficient file processing and data manipulation.

11. Future Scope

• Integrate support for analyzing binary file metadata.
• Develop a graphical user interface (GUI) for easier file selection and result visualization.
• Add functionality to compare metadata across multiple files and generate comparative reports.

12. References

• Node.js Documentation: https://nodejs.org/en/docs/
• MDN Web Docs - JavaScript: https://developer.mozilla.org/en-US/docs/Web/JavaScript
• Regular Expressions Guide: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions

13. Annexure

• Internship Completion Certificate
• Offer Letter